

Overview

AI codebases decay quickly without clean code, modular boundaries, and documentation. Notebook-first experimentation is useful, but production reliability requires explicit architecture.

Clean Code Principles

1. **Meaningful naming:** `train_classifier` is better than `run1`.
2. **Single responsibility:** each function handles one coherent task.
3. **Small interfaces:** pass explicit arguments, avoid hidden globals.
4. **Avoid duplication:** centralize shared preprocessing logic.
5. **Fail loudly:** validate assumptions and raise helpful errors.

Modular Design

Recommended decomposition: - `data/` ingestion and validation. - `features/` transformation logic. - `models/` training/prediction APIs. - `evaluation/` metrics and reporting.

Benefits: - Better testability. - Easier parallel development. - Lower blast radius for changes.

Documentation Practices

Three layers: 1. **Docstrings:** what function does, args, return, exceptions. 2. **README:** setup, run steps, project intent. 3. **Architecture notes:** data flow and system boundaries.

Documentation should describe assumptions and limitations, not only happy path.

AI-Specific Examples

- Move feature engineering from notebook cells to `feature_pipeline.py`.
- Expose `train_model(config)` and `predict(records)` stable interfaces.
- Keep experiment scripts separate from reusable package code.

Common Pitfalls

- Monolithic notebooks mixing ingestion/training/evaluation/deployment.
- Hardcoded file paths and secrets.
- Unclear naming (`tmp`, `data2`, `final_final`).
- Lack of tests for data contracts.

Business Case Studies & Exceptions

Case 1: Feature Drift from Duplicate Logic

Two teams reimplemented feature transform differently; online/offline mismatch caused prediction quality drop.

Fix: - Single shared feature module. - Contract tests to compare outputs across environments.

Case 2: Incident Escalation Blocked by Missing Docs

On-call engineer could not identify data source ownership because pipeline had no architecture docs.

Fix: - Add service/component README. - Add runbook with alert meanings and owner contacts.

Interview Questions & Answers

1. **Q: What is single responsibility principle?**
A: Each unit should have one reason to change, reducing coupling and simplifying tests.
2. **Q: Why modularize ML pipelines?**
A: Reuse components, isolate failures, and enable independent iteration.
3. **Q: What belongs in function docstring?**
A: Purpose, args, return type, raised exceptions, and edge-case behavior.
4. **Q: Why avoid hardcoded paths?**
A: They break portability across local, CI, and production environments.
5. **Q: How do clean code practices reduce incidents?**
A: Clear logic and naming speed debugging and reduce accidental side effects.
6. **Q: Unit vs integration test?**
A: Unit test isolates small component; integration test checks interactions between components.
7. **Q: How refactor notebook into modules?**
A: Extract repeatable logic into functions/files, leave notebook for orchestration and analysis narrative.
8. **Q: What is code smell in ML repo?**
A: Copy-pasted preprocessing across scripts yielding inconsistent train/inference behavior.
9. **Q: Why document assumptions?**
A: Hidden assumptions cause silent failures when data or business context changes.
10. **Q: How does modularity help CI/CD?**
A: Targeted tests per module and safer incremental releases.